Audelle: Measurement-Grounded Machine-Learning Audio Editing on Commodity CPU Hardware

Lohit Aksha

Abstract—Audio editors increasingly ship "AI" features whose behaviour is asserted rather than measured. We report Audelle, an editor in which every learned component is accompanied by a quantitative claim, a ground-truth fixture that supports it, and an explicit statement of what it cannot do. The system runs entirely on a mobile CPU with no accelerator. It combines per-request model selection for music source separation, statistical-model speech enhancement, energy/spectral-flatness voice activity detection with clustering-based speaker diarization, a timbre network trained on 3 856 real instrument notes, a phrase model learned from 1 857 annotated melodic intervals, and repetition-based audio inpainting. Natural-language requests are routed by a deterministic grammar rather than a language model, so an edit cannot be hallucinated. We measure the system against synthetic ground truth throughout: distortion-to-noise ratio for speech enhancement, onset alignment error for beat-locked layer placement, scale-invariant signal-to-distortion ratio for inpainting, and per-clip accuracy for genre classification. Evaluation exposed defects that code review had not, including a source separator that made speech worse, a voice activity detector that returned zero segments on contiguous speech, and a melodic model whose regression head collapsed all intervals to a single near-zero value. Three evaluation metrics were themselves found to be invalid and were discarded or renamed rather than reported; one speech-enhancement figure was revised downward by a factor of two once its metric was replaced. A 99-case prompt benchmark and 328 automated tests guard the resulting behaviour. We argue that publishing negative results and invalidating one's own metrics is a necessary part of deploying learned audio components in interactive tools.

Index Terms—music information retrieval, audio source separation, speech enhancement, speaker diarization, audio inpainting, evaluation methodology, reproducible research.

I. Introduction

A. The problem with unmeasured audio ML

Modern audio editors advertise natural-language editing, stem extraction, voice cleanup and "seamless" fill. In practice these features are frequently demonstrations rather than mechanisms: a crossfade labelled as inpainting, a pitch-swept sine labelled as a drum, a randomly populated bar chart labelled as a stem analyser. Such interfaces are worse than no feature at all, because they transfer the verification burden to the user.

Audelle is an audio editor whose design constraint is that no user-visible behaviour may be shipped without a measurement behind it. Where a component cannot be validated, it is relabelled or removed. This paper describes the resulting system and, equally importantly, the evaluation methodology that made it trustworthy, together with the defects that methodology uncovered.

B. Contributions

1) A CPU-only audio editing architecture that selects source-separation models per request rather than globally, on the principle that a music-trained model must not process speech (Section III-B).
2) A timbre network trained from scratch on 3 856 real notes, paired with a deterministic additive renderer, following the hybrid differentiablesignal-processing design (Section III-E).
3) A phrase model that predicts where rests fall, with the number of rests supplied by a tempo-and-energy-aware arrangement model (Section III-F).
4) Repetition-based inpainting that reconstructs a region and preserves the timeline, replacing a crossfade that deleted audio (Section III-G).
5) A measurement methodology (Section IV) built on synthesised ground-truth fixtures, and an account of the two evaluation metrics it invalidated and two of the six benchmark families it caught regressing.

C. Scope

Audelle is an editing tool, not a music generator. Large text-to-audio generation was considered and rejected: it is the least differentiated component against dedicated commercial products, the most expensive to run on the target hardware, and orthogonal to Audelle's purpose. Where the system adds musical material, that material is rendered by Audelle's own trained timbre model and its own recorded drum bank.

II. Related Work

A. Source separation

Time–frequency masking with an ideal ratio mask remains a strong baseline, and Conv-TasNet showed that learned time-domain masking can surpass it [1]. Spleeter packaged fixed pretrained models for fast separation [2], and Open-Unmix provided a reference implementation and dataset tooling for the same task [3]. Hybrid Transformer Demucs replaced the innermost bi-U-Net layers with a cross-domain transformer using self-attention within a domain and cross-attention across domains, reporting state-of-the-art results on MUSDB18

with additional training data [4]. Audelle uses Demucs, but its contribution here is a routing policy: only htdemucs_6s contains piano and guitar sources, so requests naming those instruments cannot be served by the general model.

B. Speech enhancement

Statistical-model methods, in particular the Wiener formulation and its decision-directed descendants, remain the reference against which learned enhancers are measured [5]. Audelle implements the classical decision-directed Wiener estimator with over-subtraction and a spectral floor, selected because it requires no training data and degrades gracefully on clean speech.

C. Transcription and diarization

Whisper demonstrated that large-scale weak supervision yields competitive zero-shot recognition without fine-tuning [6]. Voice activity detection by adaptive energy thresholding with hangover, and clustering of MFCC statistics, are long-established techniques [7]; selection of the cluster count by the silhouette coefficient [8] follows Rousseeuw. Audelle implements this stack directly rather than depending on a neural diarization pipeline, because the target hardware has no accelerator and the pipeline must start in under a minute.

D. Generative timbre

Differentiable digital signal processing established that a synthesizer's control parameters can be learned while its rendering stage remains hand-written [9], and the design has since been reviewed and extended across music and speech [9]. Bazin et al. cast note generation as spectrogram inpainting, using a vector-quantised spectrogram autoencoder with a masked transformer decoder [10]. Audelle adopts the hybrid philosophy of the former but not the generative model of the latter: a small network predicts spectral envelopes and envelope parameters, and an additive synthesizer renders audio, so no inference-time vocoder is required.

E. Repetition structure

Rafii and Pardo's REPET exploited the repetitious structure of music to separate vocals from accompaniment without a trained model [11]. Audelle borrows the same observation for a different task: if a region must be reconstructed, the best available material is often the same musical position played elsewhere in the track.

F. Classification and features

Mel-frequency cepstral coefficients remain a strong general-purpose acoustic representation [7]. Genre classification from spectral, chroma, energy and tempo features is a standard benchmark [12], and the random forest ensemble remains a strong tabular baseline [13]. Audelle uses a permuted-temperament key profile for key matching [14] and log-mel features for repetition search. All signal processing is performed with librosa [15].

III. System Architecture

A. Execution model

Audelle is a Next.js front end with a FastAPI backend. Requests are resolved by a two-stage planner. A deterministic grammar parses the utterance into an intent plus typed parameters; the executor then selects the algorithm. Prompts that match no grammar rule are rejected with an explanation rather than approximated. This is a deliberate design choice over a large language model: the edit space is finite and enumerated, so a generative router can only introduce failures. A latent language-model path is retained in the codebase and is inert because no API key or local model is configured; it is documented as dormant rather than deleted.

B. Per-request separation model selection

Separation is chosen by the requested source, not globally. A general four-stem model (htdemucs_ft) serves vocals, drums and bass. Piano and guitar requests route to htdemucs_6s, the only available model containing those stems. Source aliases are normalised through a resolver that accepts natural phrasing; an instrument with no representable stem is rejected explicitly rather than force-fitted, because silent coercion was the cause of a shipped defect in which "remove all instruments and keep the voice" routed to vocal removal.

A second routing rule follows from domain match: speech is never processed by a music-trained separator. Section V-A reports the measurement that established this rule, which reversed the system's original design.

C. Statistical speech enhancement

The denoiser estimates the noise floor as a per-bin low percentile of the short-time magnitude spectrum, forms an instantaneous SNR from that floor, and applies a Wiener gain with a controlled over-subtraction factor and a spectral floor, smoothed in time to suppress musical-noise pumping. A noise-aware presence lift compensates for the dullness left by broadband gain reduction, gated per bin so that bins already dominated by noise are not boosted. The algorithm requires no training data and is invariant to the recording microphone. The floor is a single file-wide percentile and therefore assumes a stationary noise floor; Section VIII records the failed attempt to make it adaptive.

D. Voice activity detection and diarization

Frames are classified active when short-time energy exceeds an adaptive floor defined in the log domain as

$$\tau = L_{\text{noise}} + \max(5,\ 0.25\,(L_{90} - L_{10}))\,, \qquad (1)$$

where L_{10} and L_{90} are the 10th and 90th percentiles of frame energy in dB. Segments shorter than 0.5 s are discarded and 0.25 s of hangover is applied, so that pauses inside a speaker turn do not split it. Each segment is summarised by the mean and standard deviation of its

MFCCs, and k-means partitions these; k is chosen by the silhouette coefficient. Adjacent segments assigned to the same cluster are merged. Section V-B reports why the original threshold returned zero segments on contiguous speech.

E. Learned timbre with a deterministic renderer

A note is parameterised by a 32-band log-spectral envelope sampled over its sustain, plus attack time, decay ratio, spectral centroid and spectral flatness. The network TimbreNet maps an instrument embedding of dimension 24, concatenated with normalised pitch and velocity, through two hidden layers of 128 units to the 36-dimensional output. Partial amplitudes are read off the predicted envelope with an inharmonicity term; flatness drives a noise layer and the predicted parameters drive an ADSR envelope. Trained on 3 856 notes spanning 27 instrument-family and source combinations, with validation loss falling from 0.946 to 0.685, this is deliberately a hybrid: the timbre is learned, the renderer is ordinary additive synthesis, so a wrong prediction produces a poor note rather than garbage. Instruments outside the trained set return no prediction and fall back to a labelled DSP synth; no fuzzy nearest-family match is attempted, for the same reason instrument resolution is not.

F. Phrase-level melodic model

The phrase model learns the distribution of melodic intervals and durations by classification into bins, not by regression. Both the head formulation and the arrangement stage are described below.

a) Classification, not regression.: An earlier head regressed the interval and duration directly. A regressor trained under a conditional mean loss returns the mean of its target distribution, which for a sparse interval distribution is near zero; the measured mean absolute interval was 0.00 semitones and the generated lines were monotone. The model now classifies the interval into 15 bins spanning -7 to $+7$ semitones, the duration into 16 bins of which the zeroth denotes a rest, and predicts velocity directly, with the architecture $5 \to 96 \to 96 \to 32$. Section V-C reports the before-and-after statistics.

b) Rests.: The model predicts the placement of rests; the arrangement stage decides the number, by blending the model's rest odds with a target fill rate as a geometric mean. This split avoids the degenerate case in which the model, having learned that rests are rare, emits none, and avoids the opposite failure in which rests are inserted at random. Planning precedes rendering so that a note immediately repeated at the same pitch is merged rather than re-struck, and an empty plan is guarded against rather than rendered as silence.

G. Rhythmic and tonal conditioning

Detected tempo locks all generated layers: the drum sequencer places sixteenth notes on the measured beat grid, and generated notes are snapped to the detected beat phase. Detected key conditions melodic content. For two-file jamming, keys are matched by correlating chroma histograms against Krumhansl–Kessler key profiles [14]; alignment is refined by maximising onset-envelope cross-correlation; and levels are matched by RMS.

H. Recorded drum bank and rule-based arrangement

Every drum hit Audelle plays is a recorded sample: 1 200 one-shots across ten percussion types, extracted and de-clicked at build time and stored as FLAC, 34.7 MB in total. The previous implementation synthesised kicks from a pitch-swept sine and snares from band-passed noise, which is why the "add drums" feature sounded synthetic.

The distinction matters for what may be claimed. The samples are real recordings; the arrangement — which hits land on which sixteenth-note step — is a hand-written pattern library of named styles, tempo-synced to the measured beat grid. No trained drum model exists or is claimed: nothing in the system generates or renders a drum hit with a neural network. A test verifies that the instrument catalogue advertised by the user interface matches the instruments the backend can render, in both directions, so a style cannot be promised and then found missing.

A separate random forest may select one of those hand-written patterns. It sees eight rhythm features — tempo, four-on-the-floor ratio, half-time ratio, swing, hat density, sub-bass ratio, kick rate and snare rate — and predicts one of fifteen feels. The prediction is used only when its confidence exceeds 0.45, and a style named explicitly in the prompt always overrides it. The classifier therefore chooses among existing arrangements; it never synthesises audio, and its accuracy is not evidence that the arrangement itself is learned.

I. Repetition-based inpainting

Given a region to remove, the system seeks replacement material from the same track, in three escalating strategies:

1) Bar-aligned repeat. If the region damages bar 9, the correct material is bar 5 or bar 13 — the same musical position. The existing beat detector supplies the grid, so candidate positions one or more bars before the hole are enumerated directly, and each is scored by log-mel similarity to the audio on either side of the hole.
2) Feature search. A sliding search over the remaining track, scored identically, for material that is similar without being bar-aligned.
3) Spectral fill. Log-magnitude interpolation between the flanking frames, with phase propagated from the preceding frame's instantaneous frequency and corrected to land on the following frame.

The timeline length is preserved, because a request to remove a transient artifact means to keep the music, not

to shorten the file. The metadata reports which strategy ran, the source offset, and the match distance, so that a fallback is never presented as a successful repeat.

IV. Measurement Methodology

A. Principle

No claim is reported unless a measurement supports it, and every measurement scores against ground truth whose value is known in advance. Synthetic fixtures are constructed so that the correct answer is available by construction: a loop built from a fixed period so that the phase-correct repeat exists in the file; speech mixed at a known SNR so that the improvement is computable; layer generated against a known grid so that alignment error is exact.

Reconstruction quality is scored with the scale-invariant signal-to-distortion ratio [16], chosen because it is invariant to the overall gain mismatch between an estimate and its reference and therefore cannot be improved merely by scaling the output — a failure mode that plain SDR invites. Metrics are chosen to be unfakeable: no metric in this paper rewards silence, since an estimator that emits nothing would score infinitely well on several otherwise tempting measures.

B. Prompt benchmark

A 99-case benchmark covers every intent the grammar supports, including adversarial phrasings: “phonk” must not be captured by a rule matching “honk”; “cut pauses” must not be captured by a trim rule; “0.5 seconds” must not be parsed as a duration from the wrong clause. All 99 cases pass.

C. Automated regression suite

293 automated tests execute. Beyond ordinary unit tests, the suite contains ground-truth regression tests whose assertions are statements about measured physics: that phrase statistics fall within measured bounds, that jam alignment median error is below a threshold, that the filled region of an inpainted file is not silent, and that the instrument catalogue advertised by the user interface matches the instruments the backend can render.

V. Results

A. Domain-matched separation (negative result)

Routing speech through a music-trained separator degraded quality. On three broadcast-speech fixtures the SNR change ranged from $-4.3\,\mathrm{dB}$ to $-7.2\,\mathrm{dB}$. After replacing that path with a Wiener filter and noise-aware presence lift, the same fixtures improved by $+0.3$ to $+0.7\,\mathrm{dB}$ true SNR across the three strength settings, with the voice level raised 1.9–2.9 dB and already-clean speech returned byte-identical.

These figures were revised downward after the metric itself was corrected. The earlier report quoted $+1.3$, $+10.1$ and $+2.7\,\mathrm{dB}$, but those were produced by a file-level proxy comparing the 90th-percentile frame against the 10th. That quantity rises when a denoiser attenuates quiet frames, which is not the same as improving signal-to-noise ratio. Measured against the known clean reference, the proxy overstated the improvement by roughly a factor of two; the field was renamed dynamic_range_gain_db and the metadata now reports the noise-floor drop and voice-level lift that are directly measurable at inference. Section VII records this as the second invalidated metric.

The honest conclusion is that this filter is a gentle cleanup, not a denoiser: it reliably makes speech more audible against a steady noise floor and cannot make worse speech worse, but half a decibel is a small effect and no listening test was conducted. This result reversed the system’s routing design. It is also the reason synthesised speech was rejected as a test input: synthesised voices fall outside the separator’s training domain and yield near-silent stems, which would have produced a spuriously excellent score.

B. Adaptive VAD (negative result)

The original threshold was $r > p_{40}(r) \cdot 1.5$, which fails whenever speech occupies more than roughly 60 % of a file: the 40th percentile then lies mid-speech and the multiplier excludes real speech, yielding zero segments. Equation (1) removed this failure mode. Two further defects surfaced only under test: cluster selection required at least $k + 2$ segments, so a three-turn exchange could never resolve two speakers, and the span-removal routine skipped any span beginning at sample 0, so leading audio survived every “keep only speaker 1” request.

C. Phrase model (quantitative)

Across 13 annotated tracks yielding 1 857 interval–duration pairs, the change from regression to classification produced the statistics in Table I.

TABLE I
Measured phrase statistics, 13 tracks, 1 857 pairs.

Statistic	Regression	Classification
Mean $\lvert\Delta\rvert$ (semitones)	0.00	4.69
Distinct pitches per line	5	10
Longest same-pitch run	15+	2

A further defect was caught only by measuring the loudness of quiet tracks: a state slot conflated per-note amplitude with track energy, so quiet inputs rendered as total silence. The same measurement revealed a fill density of 10 %, against which the arrangement model was subsequently calibrated.

D. Repetition-based inpainting

On a fixture with a known 2 s period and a hole punched into one repetition, duration was preserved exactly (8.00 s, against 7.50 s for the previous splice-out) and the filled region was non-silent. We explicitly do not claim that

the original notes are recovered: on through-composed material they cannot be, because that audio no longer exists in the file. Seam continuity is likewise not claimed as an improvement, since a 50 ms equal-gain crossfade can post a smaller single-sample step than the reconstructed splice; the tests assert an upper bound rather than superiority. An intermediate design that minimised seam discontinuity as its refinement objective was discarded after measurement, because the objective is minimised at any quiet point and the matcher consequently drifted away from the phase-correct repeat.

E. Beat-locked drum placement

Because all drum hits are now recordings, placement can be scored directly. The energy of the onset-envelope restricted to the kick band was measured on-grid against off-grid over generated patterns. On-grid placement yields 1.97× the kick-band onset energy of off-grid placement, which confirms that the sequencer is genuinely locked to the measured tempo rather than merely producing something rhythmically plausible.

F. Key-aware two-file fusion

When a user supplies a second recording, the two are fused rather than merely concatenated. Keys are matched by correlating chroma histograms against Krumhansl–Kessler key profiles [14], so a track in A minor is fused against one in B minor by transposition rather than by luck. The beat phase is then aligned by maximising cross-correlation of the two onset-envelope functions, and levels are matched by RMS. Alignment was measured on synthetic fixtures of known tempo and key — 120 BPM A minor against 140 BPM D minor — and reaches a median onset-alignment error of 0.0 ms with a 90th percentile of 11.6 ms, which is below the resolution at which the discrepancy is audible as a flam.

G. Genre classification

The classifier is a random forest [13] over 75 features extracted per clip: the mean and standard deviation of 20 MFCCs and of a 12-bin chromagram, of spectral centroid, bandwidth, rolloff, zero-crossing rate and RMS, together with estimated tempo. Training uses a balanced subset of 100 clips spanning the ten GTZAN genres [12]. Results are reported from a dedicated evaluation script with a fixed seed, which writes its output to a JSON artefact in the repository; the earlier training script printed its numbers to stdout and discarded the audio, so nothing on record stated what the shipped artefact actually scored. Table II summarises the result.

TABLE II
Genre classification, 100 GTZAN clips, 10 classes, 75 features, fixed seed. Chance is 10 %.

Protocol	Accuracy	Clips
5-fold stratified CV	75.0 ± 8.9 %	100
30 % holdout	76.7 %	30
Chance	10.0 %	—

Both protocols agree at roughly 7.5 times chance, which is consistent with the literature on spectral and timbral genre cues and with the dataset size. The per-fold spread of 60–85 % is wide relative to the mean, which is the expected behaviour at this sample size and the reason the standard deviation is reported rather than suppressed. Per-class holdout accuracy is in server/ml/models/genre_eval.json; it ranges from 100 % for country, disco, metal and reggae to 33 % for pop, which is the class the feature set distinguishes least. With 100 clips, no comparison against published genre-classification results is meaningful, and none is made: the purpose of this evaluation is to establish that the shipped artefact is better than chance and to make that number reproducible, not to enter a benchmark.

H. Prompt routing and regression

All 99 benchmark cases pass and all 293 tests pass. The benchmark is the more informative artefact: it encodes routing failures that unit tests would not catch, including two cases where a shorter rule shadowed the correct one.

I. Hardware envelope

The entire pipeline runs on a mobile 8-core CPU. The timbre model trains in roughly 30 s and the drum bank builds in roughly 1 min. Source separation is the dominant cost of any request that invokes it.

VI. Threats to Validity

The system was developed and evaluated on a single class of machine: a mobile 8-core CPU with 15.3 GB of RAM and no GPU. All reported accuracies, energies and alignments are therefore wall-clock-bound and are not claims about throughput on accelerator hardware. Small-sample evaluation is a genuine limitation: the genre classifier is trained on 100 clips, which is two orders of magnitude smaller than a competitive benchmark, and the reported figures should be read as a floor rather than an estimate of attainable accuracy. Ground-truth fixtures are synthesised and therefore stylistically narrow; they are strong for verifying numerical claims and weak for verifying perceptual ones. Finally, no human listening test was conducted. Subjective quality claims are deliberately absent from this paper.

VII. Failure Modes and Invalid Metrics

We report two occasions on which our own evaluation was unsound, because both would have produced favourable published numbers.

a) Synthetic speech against a music-trained separator.: Evaluating separation on synthesised speech produced near-silent stems and an excellent-looking score. Synthesised voices are outside the separator's training domain; the metric was measuring the input, not the system. All separator evaluation was moved to real recordings.

b) Comparing two different separation models against each other.: Comparing the output of one separation model to the output of another produced an apparent improvement of −106 dB. The two models were being scored against each other rather than against ground truth, and the number was an artefact of mismatched assumptions. The comparison was discarded. After this, no metric is reported without ground truth of known value.

c) A dynamic-range proxy reported as SNR.: The speech-enhancement figure was computed with broadband_snr_db, which compares the 90th-percentile frame against the 10th and is therefore a measure of dynamic range, not of signal-to-noise ratio. A denoiser attenuates quiet frames, so the proxy rises substantially even when true SNR barely moves. Measured against a known clean reference the proxy overreported the improvement by roughly a factor of two. The metric was renamed and the reported quantity replaced with the noise-floor drop and voice-level lift, which are directly observable without a reference signal. True SNR is not computable at inference at all, because it requires the clean audio that the input no longer contains.

d) Exceptions that hid errors.: Two bare except: pass handlers silently swallowed genuine exceptions — a NameError in the timbre layer and a wrong keyword argument in the denoiser — after which the feature fell back to a default while still reporting success. Failure is now surfaced in returned metadata, in the manner of the existing phrasing_error field.

e) Licensing.: The drum bank is built from 1 200 recorded one-shots obtained from a public repository that does not declare a licence. We were unable to establish permissive terms and regard the current redistribution as unresolved. This is reported rather than silently shipped.

VIII. Limitations and Future Work

The phrase model is trained on 13 tracks, which is thin; the cache mechanism avoids re-running separation, so scaling the corpus is inexpensive. The groove classifier reports 87.8 % accuracy, but that figure is measured on held-out synthetic loops drawn from the same generator as its training set: it is an in-distribution result, not a real-music result, and no accuracy on real recordings is claimed. The shipped model is applied only as a confidence-gated (0.45) selector among hand-written patterns, so a wrong prediction changes which pattern plays rather than producing a wrong sound from scratch. Retraining on a labelled corpus of real songs (--songs-dir) is the next priority, and that corpus is not in the repository. Inpainting is verified for duration preservation and for the absence of silence, but its perceptual quality is not established, because ground-truth multitracks would be required. Inpainting cannot invent music that was never played, and a learned generative model — spectrogram inpainting as in [10], or an audio language model — would be required for that. Audio-to-video round-tripping is implemented: a video upload retains its original, and any edit producing audio is muxed back with the video stream copied rather than re-encoded. It was verified by demuxing the result and confirming the edited audio replaced the source. Precision sample-accurate waveform editing remains a gap relative to workstation-class editors.

IX. Conclusion

The principal claim of this work is methodological. Building an audio editor on CPU-only hardware forces every learned component to be cheap, and cheapness makes measurement easy to skip. When we measured anyway, the measurements found a separator that made speech worse, a voice activity detector that returned nothing on ordinary recordings, a melodic model that had collapsed to a constant, an inpainter that deleted audio, and two evaluation metrics of our own that were rewarding silence. A deterministic intent grammar, provenance in returned metadata, and a benchmark that encodes routing as well as arithmetic kept these from reaching users. Shipping a feature without a number behind it is cheap and invisible; shipping the number is neither, and is the only way we found to make the claim auditable.

References

[1] Y. Luo and N. Mesgarani, “Conv-TasNet: surpassing ideal time-frequency magnitude-based masking for speech separation,” IEEE/ACM Trans. Audio, Speech, Lang. Process., vol. 27, no. 8, pp. 1558–1571, 2019. doi: 10.1109/TASLP.2019.2915167.

[2] R. Hennequin, A. Khlif, F. Voituret, and M. Moussallam, “Spleeter: a fast and efficient music source separation tool with pre-trained models,” J. Open Source Softw., vol. 5, no. 50, p. 2154, 2020. doi: 10.21105/joss.02154.

[3] F.-R. Stóter, S. Uhlich, A. Liutkus, and Y. Mitsufuji, “Open-unmix — a reference implementation for music source separation,” J. Open Source Softw., vol. 4, no. 37, p. 1667, 2019. doi: 10.21105/joss.01667.

[4] S. Rouard, F. Massa, and A. Défossez, “Hybrid transformers for music source separation,” in Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP), 2023, pp. 1–5. doi: 10.1109/ICASSP49357.2023.10096956.

[5] Y. Ephraim, “Statistical-model-based speech enhancement systems,” Proc. IEEE, vol. 80, no. 10, pp. 1527–1558, 1992. doi: 10.1109/5.168664.

[6] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” arXiv:2212.04356, 2022.

[7] S. B. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” in Readings in Speech Recognition. Cambridge, MA, USA: MIT Press, 1990, pp. 85–91. doi: 10.1016/B978-0-08-051584-7.50010-3.

[8] P. J. Rousseeuw, “Silhouettes: a graphical aid to the interpretation and validation of clustering analysis,” J. Comput. Appl. Math., vol. 20, pp. 53–65, 1987. doi: 10.1016/0377-0427(87)90125-7.

[9] B. Hayes, J. Shier, G. Fazekas, A. McPherson, and C. Saitis, “A review of differentiable digital signal processing for music and speech synthesis,” Front. Signal Process., vol. 2, art. no. 1284100, 2024. doi: 10.3389/frsip.2023.1284100.

[10] T. Bazin, G. Hadjeres, P. Esling, and M. Malt, “Spectrogram inpainting for interactive generation of instrument sounds,” in Proc. Int. Conf. on AI Music Creativity (AIMC), Stockholm, Sweden, 2020, pp. 306–312. doi: 10.5281/zenodo.4285406.

[11] Z. Rafii and B. Pardo, "REpeating pattern extraction technique (REPET): a simple method for music/voice separation," IEEE/ACM Trans. Audio, Speech, Lang. Process., vol. 21, no. 1, pp. 73–84, 2013. doi: 10.1109/TASL.2012.2213249.
[12] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," IEEE Trans. Speech Audio Process., vol. 10, no. 5, pp. 302–303, 2002. doi: 10.1109/TSA.2002.800560.
[13] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001. doi: 10.1023/A:1010933404324.
[14] C. L. Krumhansl, J. J. Bharucha, and E. J. Kessler, "Perceived harmonic structure of chords in three related musical keys," J. Exp. Psychol.: Hum. Percept. Perform., vol. 8, no. 1, pp. 24–31, 1982. doi: 10.1037/0096-1523.8.1.24.
[15] B. McFee, C. Raffel, D. Liang, D. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: audio and music signal analysis in Python," in Proc. 14th Python in Science Conf. (SciPy), 2015, pp. 18–25. doi: 10.25080/Majora-7b98e3ed-003.
[16] J. Le Roux, S. Wisdom, H. Erdogan, and J. R. Hershey, "SDR — half-baked or well done?" in Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP), 2019, pp. 626–630. doi: 10.1109/ICASSP.2019.8683855.